Records GC in Collections

Dead Record Reclamation for PaginatedCollectionV2

YTDB-556

The Problem: Snapshot Isolation Creates Dead Records

When a record is **updated**, the old version is preserved so active transactions can still read it

Old version's physical location stored in the **snapshot index** (in-memory ConcurrentSkipListMap)

Once all observing transactions complete, the snapshot index entry is **evicted**

But the **physical record data on disk pages remains** — occupying space, fragmenting pages

Before this change:

Snapshot index entries were cleaned up (in-memory only)

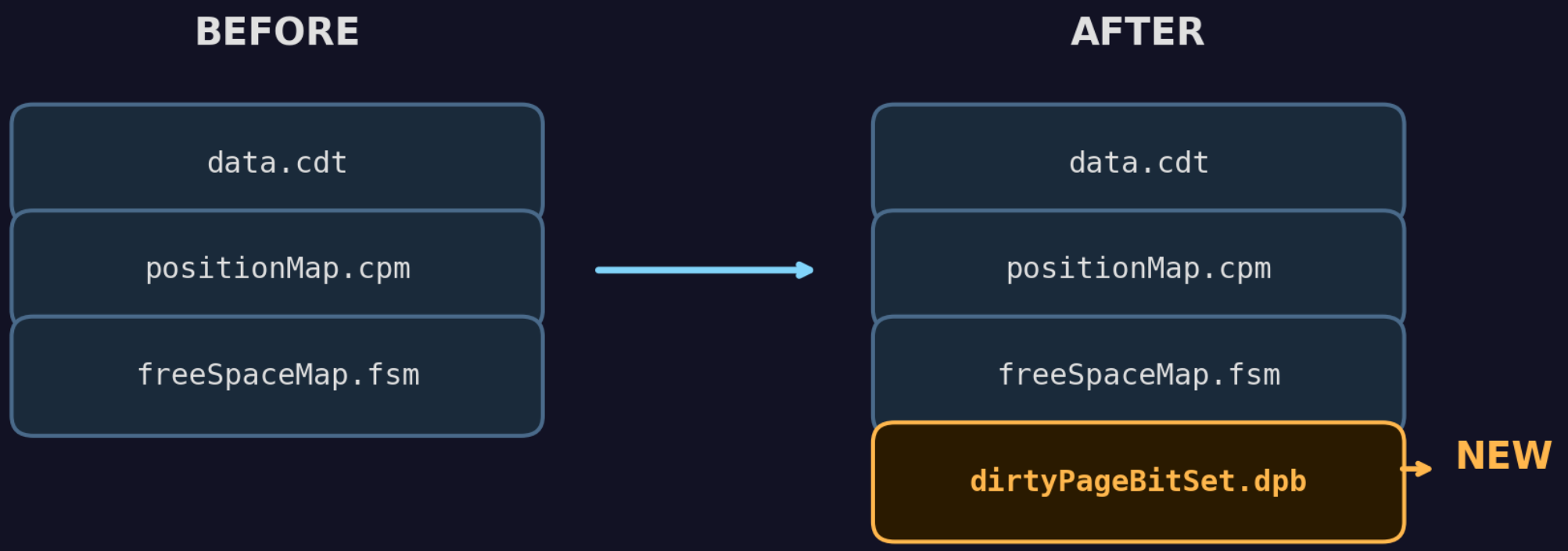
Physical record bytes on disk were **never reclaimed**

Under update-heavy workloads, collections **grow unboundedly** with dead data

Solution Overview — Three Phases

1. **Write path**: mark which pages have stale records (dirty bit set)
2. **Eviction**: count dead records per collection (trigger condition)
3. **GC cycle**: scan only dirty pages, delete dead chains, reclaim space

New On-Disk Structure: CollectionDirtyPageBitSet (.dpb)



One bit per data page in the .cdt file

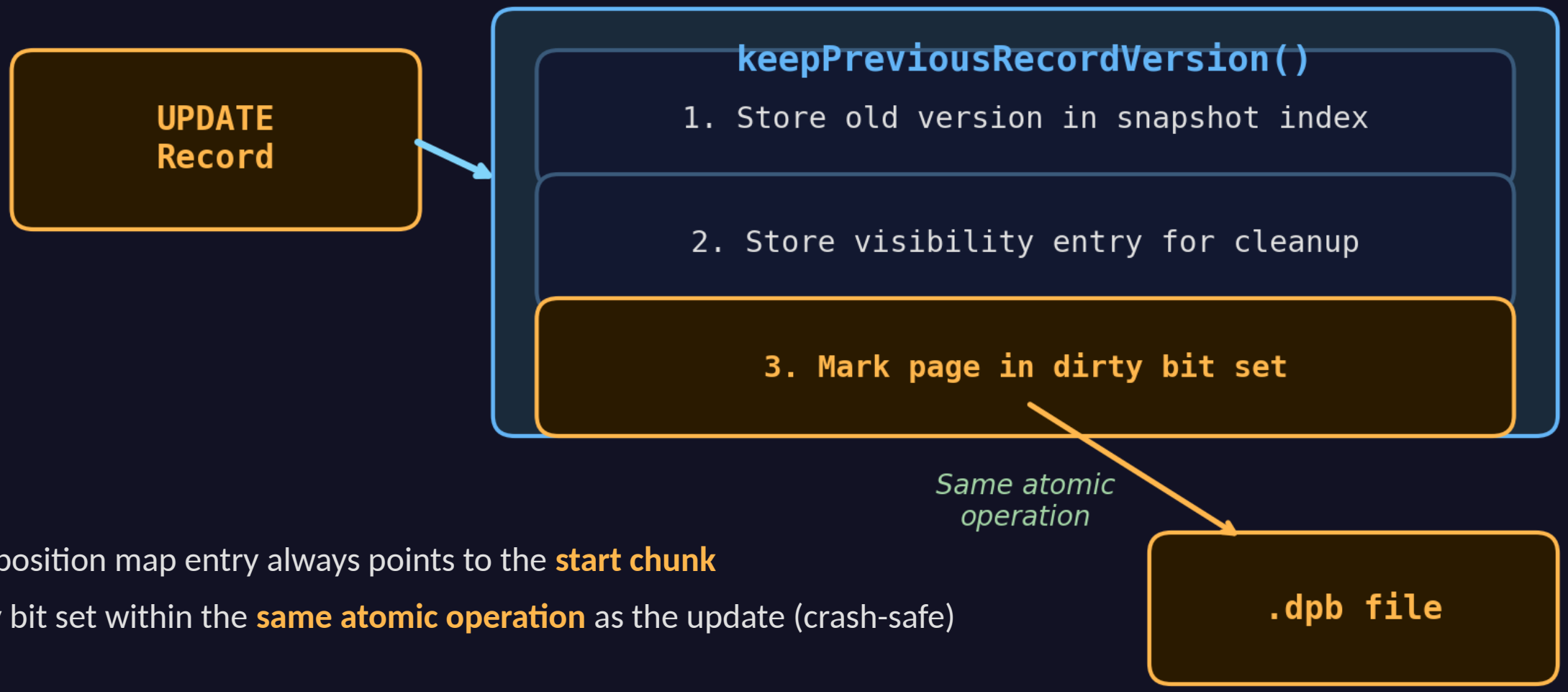
Bit = 1 means "this page has at least one stale record's start chunk"

Stored in DirtyPageBitSetPage pages (**65,312 bits per 8KB page**)

WAL-logged via AtomicOperation (**crash-safe**)

Avoids full collection scan — GC only visits dirty pages

Write Path — Marking Dirty Pages



The position map entry always points to the **start chunk**

Dirty bit set within the **same atomic operation** as the update (crash-safe)

Dead Record Counting

Per-collection AtomicLong counter: deadRecordCount

```
evictStaleSnapshotEntries()

for each stale visibility entry:

  snapshotIndex.remove(key)

  sizeCounter.decrementAndGet()

  collection.incrementDeadRecordCount()   NEW
```

→ **deadRecordCount (AtomicLong per collection)**

Incremented when a snapshot index entry is **evicted** (old version becomes truly unreachable)

Decremented (clamped to 0) after GC reclaims records

Starts at 0 on restart — pre-restart dead records aren't counted but still get reclaimed

The counter is **approximate** — good enough for triggering GC

GC Trigger Condition

When does GC run for a collection?

```
deadRecords > minThreshold + scaleFactor × approximateRecordsCount
```

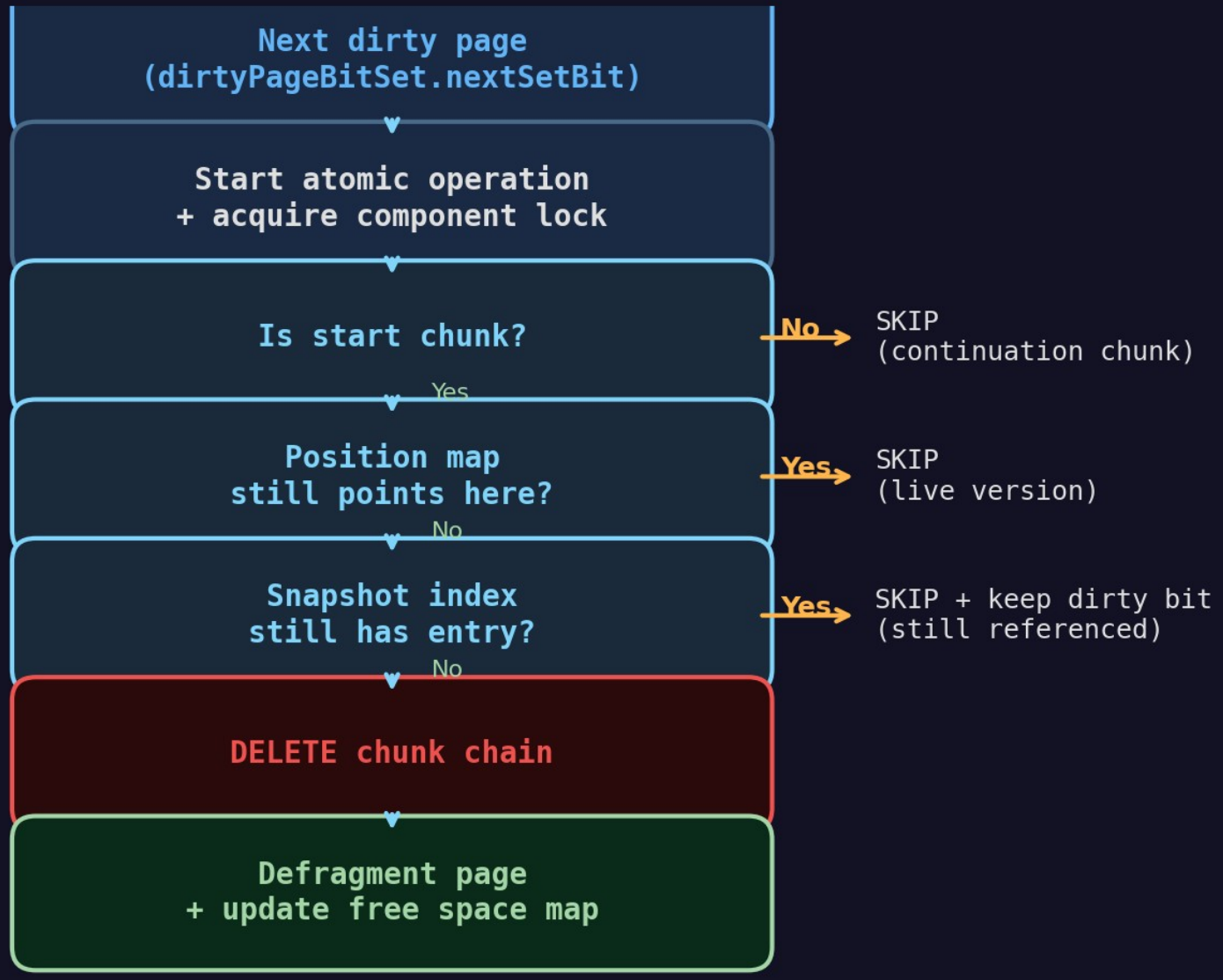
Parameter	Config Key	Default	Purpose
minThreshold	...gc.minThreshold	1,000	Avoid thrashing on small collections
scaleFactor	...gc.scaleFactor	0.1 (10%)	Scale threshold with collection size
pause interval	...gc.pauseInterval	60 sec	Delay between GC cycles

Examples:

Collection with **100 records**: triggers at > 1,010 dead records (min dominates)

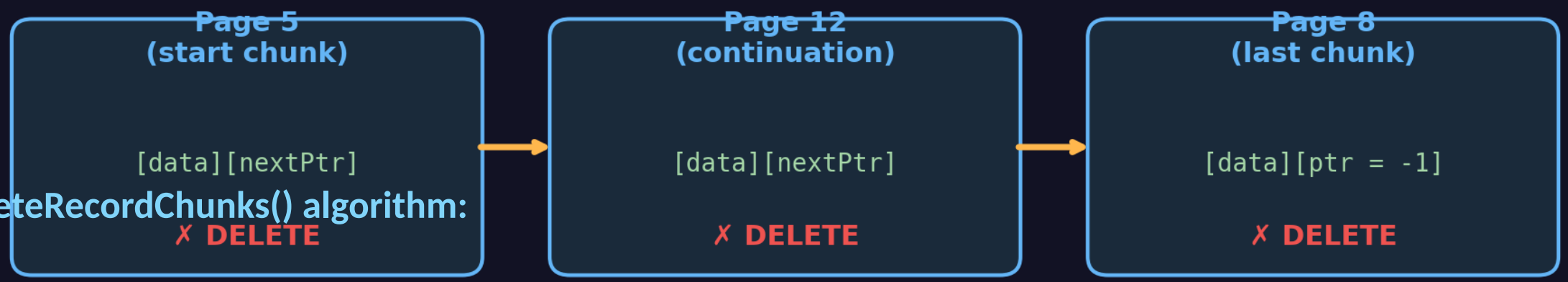
Collection with **100K records**: triggers at > 11,000 dead records (scale dominates)

Core GC Algorithm — collectDeadRecords()



Deleting a Record Chain

Records can span multiple pages (multi-chunk records):

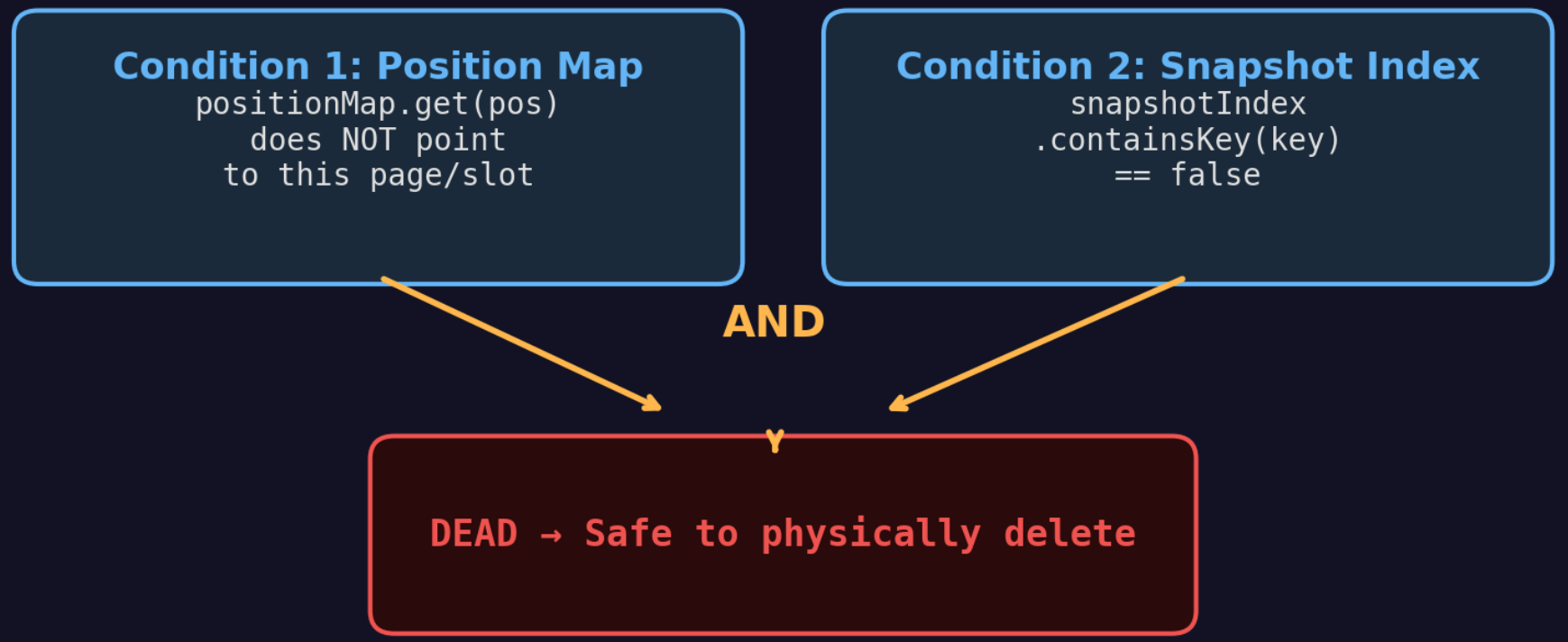


deleteRecordChunks() algorithm:

1. Read next-page pointer **BEFORE** deleting current chunk
2. Delete current chunk via page.deleteRecord()
3. Follow pointer to next page, load it for write
4. Repeat until nextPagePointer == -1
5. Collect all touched continuation pages for **batched defragmentation**

Determining If a Record Is Dead

Two conditions must BOTH be true:



If condition 1 passes but condition 2 fails: record is stale but still visible to some transaction

In this case: **anyStaleRemaining = true** → dirty bit stays set for next GC cycle

Scheduling & Error Handling

Periodic task: PeriodicRecordsGc

Runs on the **fuzzyCheckpointExecutor**

Fixed delay: configurable (default 60s)

Scheduled in DiskStorage.postOpenSteps()

Cancelled on storage close

Two-step execution:

1. Opportunistically clean snapshot indexes (tryLock, never blocks)
2. Iterate all collections, GC those exceeding threshold

Error handling:

I/O error on one page → **log, skip, continue** with next page

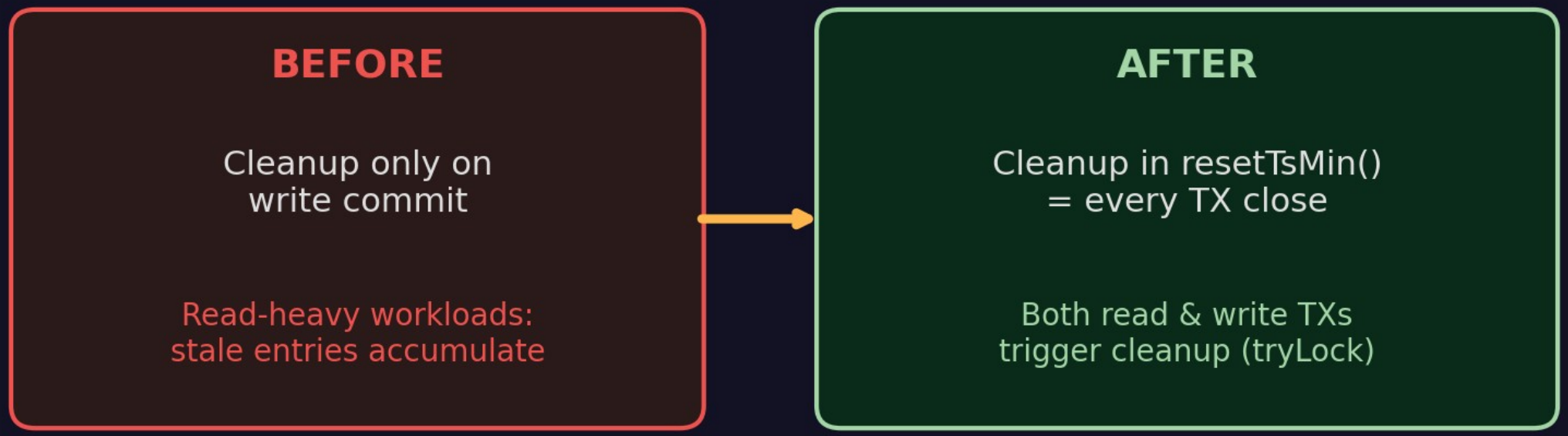
Atomic operation rolled back on failure (no partial deletes)

Per-collection errors caught and logged

One failing collection **doesn't block** others

Skip past failing page to avoid infinite retry loop

Snapshot Cleanup Moved to resetTsMin()



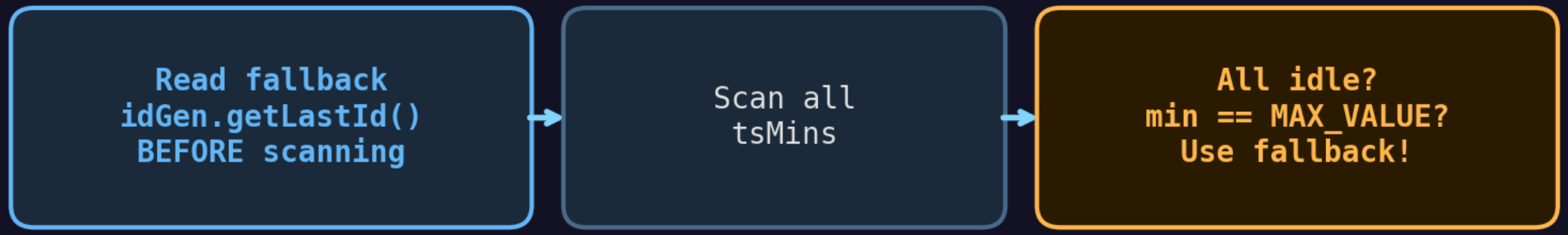
Before: snapshot index cleaned only after write commits

After: cleaned when ANY transaction closes (read or write) via resetTsMin()

Why: In read-heavy workloads with infrequent writes, stale entries accumulated

Uses **tryLock()** internally — never blocks, best-effort

Low-Water-Mark Fallback to idGen




```
Safe because: idGen is monotonic, new TX gets tsMin >= lastId,
    reading fallback BEFORE scan eliminates race condition
```

Before: computeGlobalLowWaterMark() returned Long.MAX_VALUE when no txs active → nothing evicted

After: falls back to idGen.getLastId() when idle

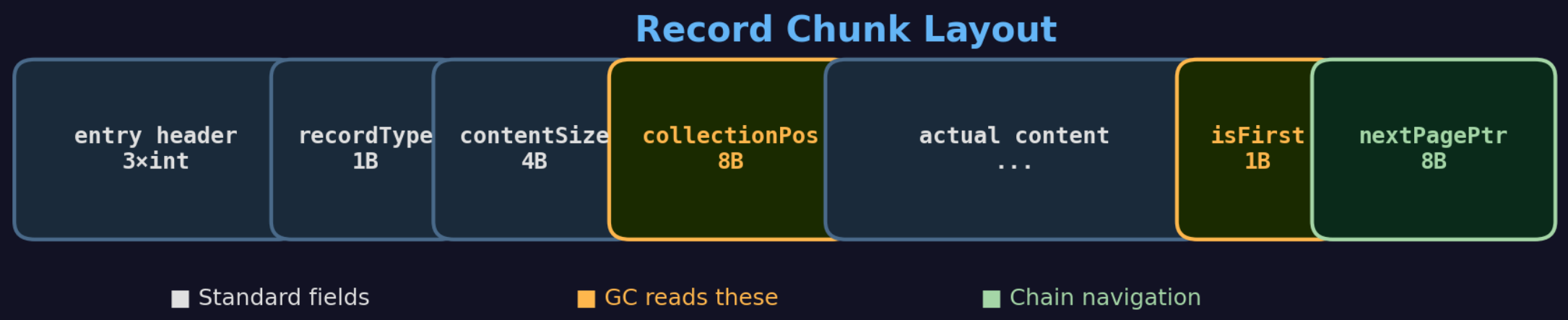
idGen is monotonic — every new transaction sets tsMin >= lastId

Reading fallback **BEFORE** scanning eliminates race with new transaction starting

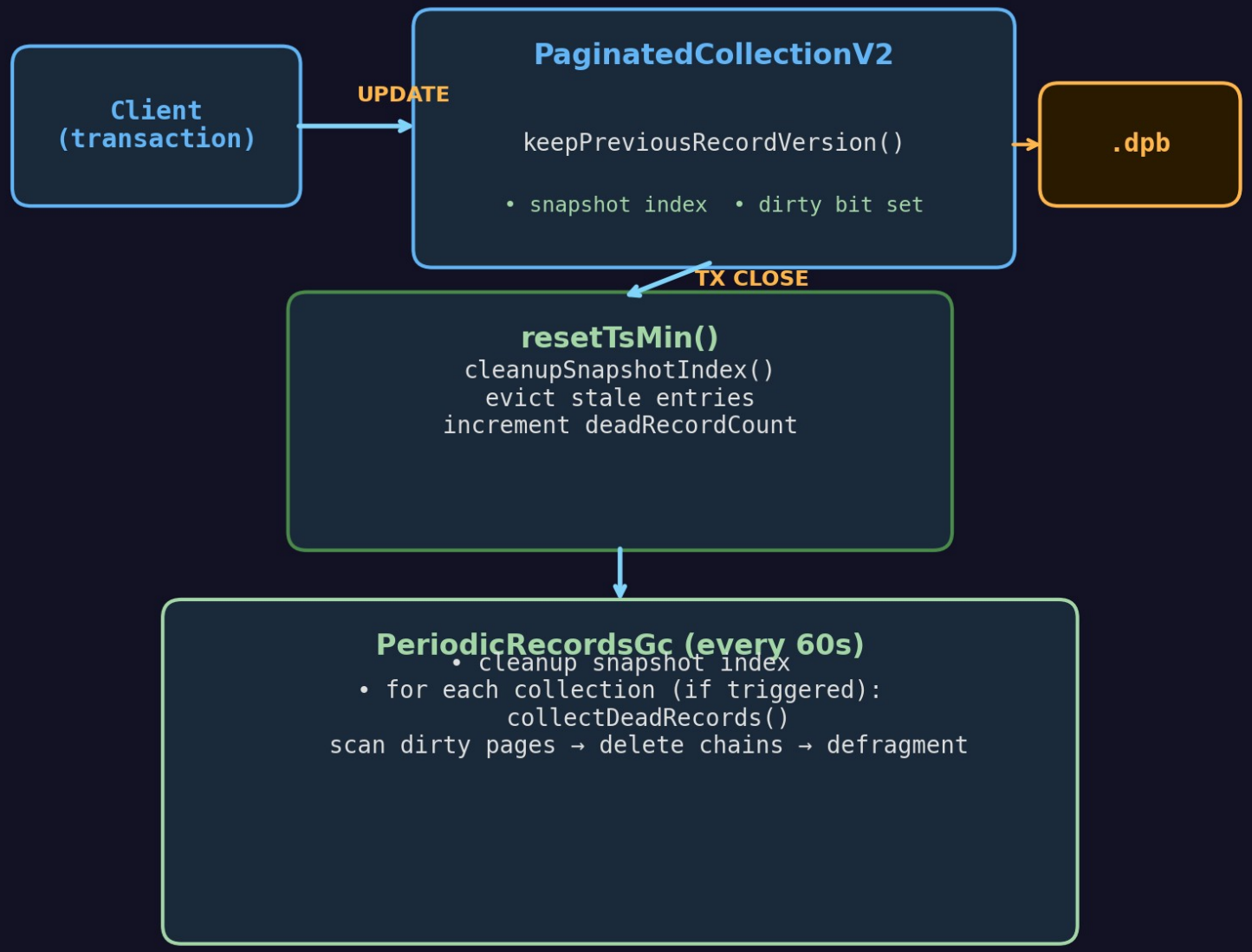
New CollectionPage Helper Methods

Method	Purpose
isFirstRecordChunk(pos)	Is this the entry-point chunk? (vs continuation)
readCollectionPositionFromRecord(pos)	Read embedded collection position from metadata header
getNextPagePointer(pos)	Read forward pointer to next chunk in chain
doDefragmentation()	Made public (was private) — GC needs to compact after deleting

These read record **metadata** without deserializing the full record content



End-to-End Data Flow



Crash Safety & Concurrency

Crash safety:

Dirty bit set mutations are **WAL-logged** (same atomic op as the update)

On crash: dirty bits may be set for clean pages → GC scans them **harmlessly**

On crash: some dead records may lack dirty bits → stay until next update marks the page

Concurrency:

GC runs inside **executeInsideComponentOperation** (exclusive lock)

Serialized against concurrent writes to same collection

tryLock on snapshot cleanup → never blocks other threads

Per-page atomic operations → small tx scope, quick rollback

Key Design Decisions

1. **Dirty bit set vs full scan** — Bit set avoids O(pages) scan; only visits pages with stale records
2. **Per-page atomic operations** — Each dirty page in its own atomic op → small WAL footprint, quick rollback
3. **Approximate counting** — AtomicLong counter is cheap, good enough for trigger; exact count not needed
4. **Batched continuation defrag** — All continuation pages from one dirty page's deletions defragmented together
5. **Cleanup moved to resetTsMin()** — Ensures prompt eviction in read-heavy workloads
6. **LWM fallback to idGen** — Prevents snapshot index from growing when system is idle

Configuration Summary

Parameter	Config Key	Default	Description
Min threshold	youtrackdb.storage.collection.gc.minThreshold	1,000	Min dead records before GC triggers
Scale factor	youtrackdb.storage.collection.gc.scaleFactor	0.1	Fraction of live records added to threshold
Pause interval	youtrackdb.storage.collection.gc.pauseInterval	60s	Delay between GC cycles

All configurable via **GlobalConfiguration** (system properties or API)

What's NOT in Scope

Tombstone (REMOVED) records — This GC handles superseded versions only, not deleted record tombstones

Index entry cleanup — Handled separately by snapshot index GC

Automatic tuning — Thresholds are static configuration; no adaptive algorithms

Questions?

Branch: ytdb-556-records-gc | Issue: YTDB-556